

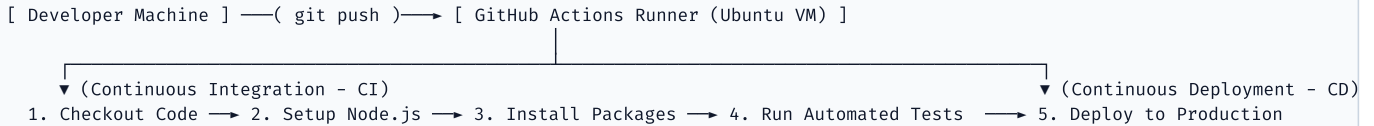


Week 1: CI/CD & GitHub Actions Master Guide

Foundations • YAML Architecture • Triggers & Filters • Quality Gates • Secrets Security • Multi-Job Pipelines

1. Core Concepts: What is CI/CD?

In modern software engineering, **CI/CD** automates the journey of code from a developer's laptop to live cloud servers.



- **CI (Continuous Integration):** Automatically tests, lints, and builds code on every commit or Pull Request to catch bugs early.
- **CD (Continuous Delivery / Deployment):** Automatically packages and deploys validated code to staging/production servers.

2. The 5 Building Blocks of GitHub Actions

Building Block	YAML Keyword	What It Does	Example
1. Workflow	<code>name:</code>	The overall automated pipeline in <code>.github/workflows/*.yml</code>	<code>name: Production CI</code>
2. Trigger	<code>on:</code>	The specific event that starts the workflow	<code>on: [push, pull_request]</code>
3. Job	<code>jobs:</code>	A set of steps that execute on a single runner	<code>jobs: run-tests:</code>
4. Runner	<code>runs-on:</code>	The virtual machine operating system provided by GitHub	<code>runs-on: ubuntu-latest</code>
5. Steps	<code>steps:</code>	Individual sequential tasks (actions or terminal commands)	<code>uses: actions/checkout@v4</code> <code>run: npm test</code>

3. Mastering Triggers, Branch Filters & Path Filters

The `on:` block controls exactly when runners should execute, saving build minutes and compute budget.

```
name: Triggers and Filters

on:
  # 1. Branch Filtering: Run on push only to main branch
  push:
    branches:
      - main
    paths-ignore:
      - '**.md'      # 2. Path Filtering: Skip workflow if only markdown changes

  # 3. Pull Request Trigger: Test code before merging to main
  pull_request:
    branches:
      - main

  # 4. Manual Dispatch: Adds a clickable "Run workflow" button in GitHub UI
  workflow_dispatch:
```

4. Automated Quality Gates & Exit Codes

GitHub Actions uses terminal **Exit Codes** to determine build status:

- **Exit Code 0** = **SUCCESS** → Step passes, pipeline continues.
- **Exit Code 1+** = **FAILURE** → Step fails, entire workflow halts immediately!

Writing an Automated Test in JavaScript (`test.js`):

```
// test.js
function add(a, b) {
  return a + b;
}

if (add(5, 5) === 10) {
  console.log("✅ Math test passed!");
  process.exit(0); // Exit Code 0: CI passes
} else {
  console.error("❌ Math test failed!");
  process.exit(1); // Exit Code 1: CI fails
}
```

⚠️ Golden Rule of CI/CD:

Never run watch mode / live reloaders (e.g. `nodemon` or `webpack --watch`) or persistent server listeners in CI! Commands must execute, finish, and exit cleanly.

5. Managing Secrets & Security

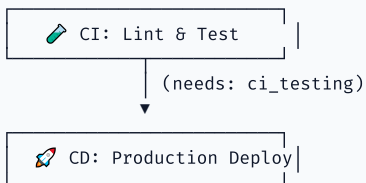
Never commit passwords, API keys, or private tokens to Git repositories. Store them in **Repo Settings → Secrets and variables → Actions**.

```
- name: Use Secret in Step
  env:
    # Injects encrypted secret into an environment variable
    API_KEY: ${ secrets.MY_API_KEY }
  run: |
    echo "Authenticating service..."
    echo "Key is: $API_KEY" # Log will display: Key is: ***
```

GitHub automatically redacts and masks all secrets as `***` in logs.

6. Multi-Job Pipelines & Dependency Graphs (`needs:`)

By default, multiple jobs run concurrently in parallel. Use `needs:` to sequence them and create a dependency graph.



7. Complete Week 1 Production Pipeline Template

```
name: Production CI/CD Pipeline

on:
  push:
    branches: [ main ]
    paths-ignore: [ '**.md' ]
  pull_request:
    branches: [ main ]
  workflow_dispatch:

jobs:
  # =====
  # JOB 1: Continuous Integration (CI)
  # =====
  ci_testing:
    name: 🟢 CI - Lint & Test
    runs-on: ubuntu-latest

    steps:
      - name: 1. Checkout Code
        uses: actions/checkout@v4

      - name: 2. Setup Node.js 20
        uses: actions/setup-node@v4
        with:
          node-version: 20

      - name: 3. Install Dependencies
        run: npm ci

      - name: 4. Run Automated Test Suite
        run: npm test

  # =====
  # JOB 2: Continuous Deployment (CD)
  # =====
  cd_deployment:
    name: 🚀 CD - Deploy to Production
    runs-on: ubuntu-latest
    needs: ci_testing
    if: github.ref == 'refs/heads/main'

    steps:
      - name: 1. Checkout Code
        uses: actions/checkout@v4

      - name: 2. Deploy using Secret Token
        env:
          SECRET_TOKEN: ${ secrets.MY_API_KEY }
        run: |
          echo "Deploying application to cloud server ..."
          echo "Authenticated Token: $SECRET_TOKEN"
          echo "🎉 Deployment Completed Successfully!"
```